

SYSTEM ARCHITECTURE

Overall Solution Architecture:

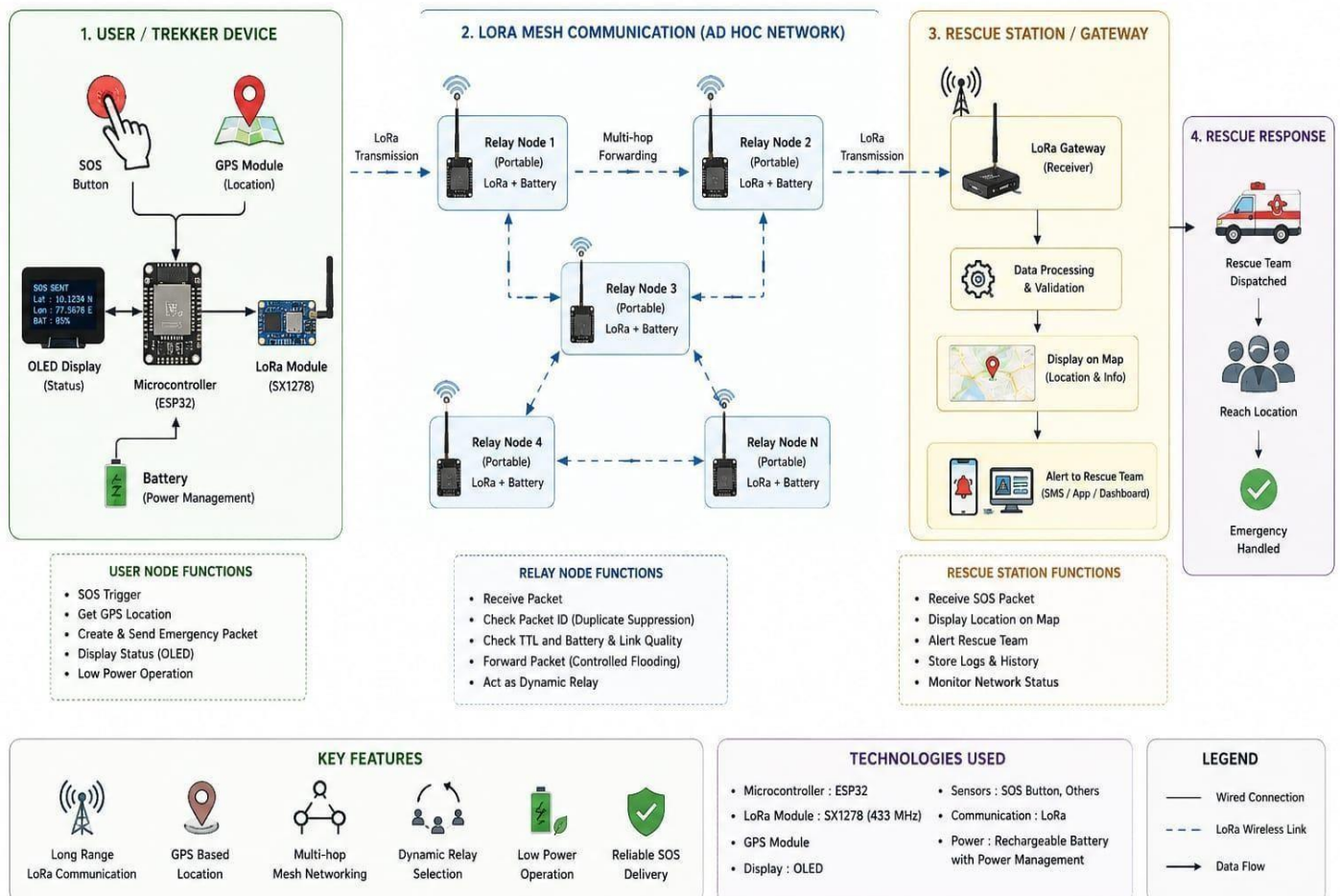
The TerraLink system follows a **multi-stage emergency communication architecture**:
Emergency Trigger → **Data Acquisition** → **SOS Packet Formation** → **LoRa Transmission** → **Relay Nodes** → **Rescue Station**

1. **Data Acquisition:** The touch sensor detects the emergency trigger, while the GPS module provides the user's location.
2. **Data Processing:** The microcontroller processes the inputs and creates an SOS packet containing the emergency and location information.
3. **Wireless Transmission:** The SOS packet is transmitted through the LoRa module.
4. **Packet Relay:** Intermediate TerraLink relay nodes receive and forward the packet until it reaches the destination.
5. **Rescue Station:** The final node receives the emergency message and location data, enabling a quick rescue response.

This architecture enables **long-range, reliable, and infrastructure-independent emergency communication**.

TERRALINK – OVERALL SOLUTION ARCHITECTURE

LoRa Based Emergency Communication System with GPS and Dynamic Relay



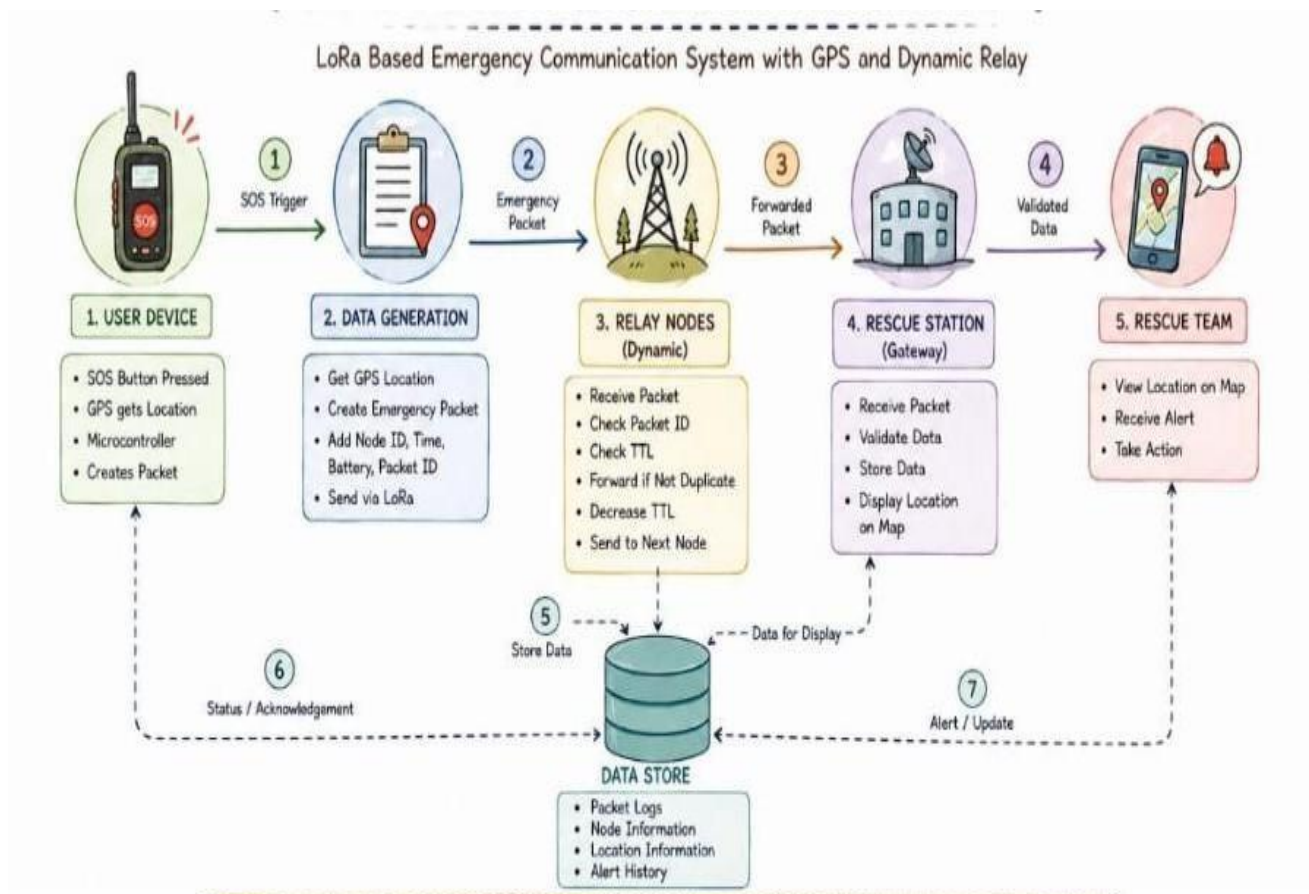
DATAFLOW DIAGRAM:

The TerraLink Data Flow Diagram represents the flow of information between the user, hardware modules, software, and communication network.

Data Flow:

User → Touch Sensor → Microcontroller → GPS Data + SOS Packet → LoRa Module → Remote TerraLink Node

The microcontroller processes the touch input, collects GPS coordinates, creates the SOS packet, and transmits it through LoRa. The received emergency information is processed and displayed on the OLED display.



HARDWARE–SOFTWARE PARTITIONING

The TerraLink system is divided into hardware and software components based on their respective functions.

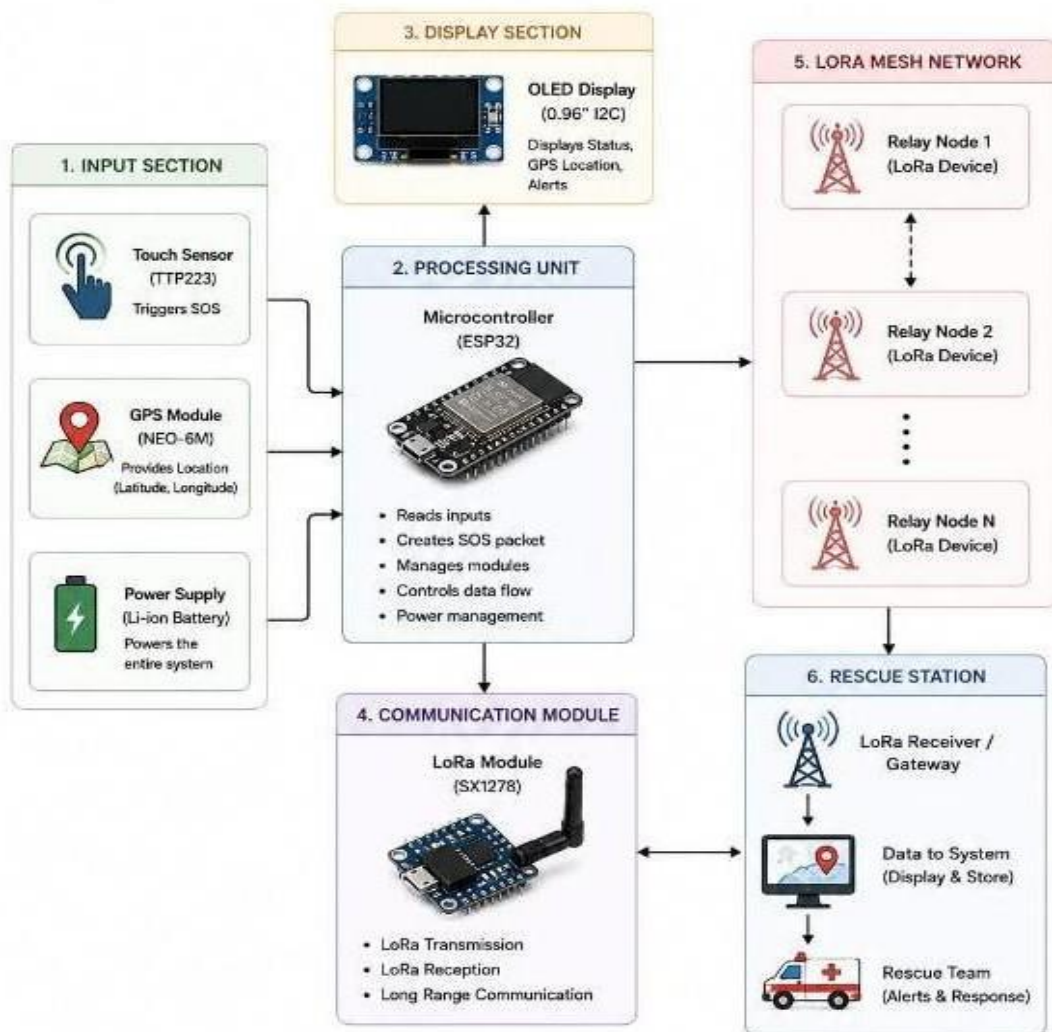
- **Hardware:** Includes the microcontroller, LoRa module, GPS module, touch sensor, and OLED display for processing, communication, location tracking, user input, and information display.
- **Software:** Handles sensor input processing, GPS data acquisition, SOS packet generation, LoRa communication, and OLED display management.

This partitioning ensures efficient, reliable, and coordinated operation of the TerraLink emergency communication syste

HARDWARE COMPONENTS			SOFTWARE MODULES		
Component	Image	Function	Module	Image / Icon	Function
Microcontroller (ESP32)		Controls all operations, processes data and manages communication.	1. Device Management		• Initialize hardware • Device ID management • Battery monitoring
LoRa Module (SX1278)		Provides long range wireless communication.	2. Sensor Management		• Read touch sensor • Detect SOS trigger • Trigger events
GPS Module (NEO-6M)		Provides location information (latitude, longitude).	3. GPS Management		• Acquire location • Parse GPS data • Provide coordinates
Touch Sensor (TTP223)		User triggers SOS alert using touch.	4. Communication (LoRa Stack)		• Create packet • LoRa transmission • Packet reception
OLED Display (0.96" I2C)		Displays status, GPS location and alerts.	5. Relay Management		• Check packet ID • TTL handling • Forward packet • Duplicate suppression
Power Supply (LI-Ion Battery)		Provides power to the entire system.	6. Data Management		• Store logs • Store location • Store alerts
Antenna (433 MHz)		Enhances LoRa signal transmission.	7. User Interface		• Display status on OLED • Show location • Show alerts
			8. Power Management		• Sleep / Deep sleep modes • LoRa power control • Optimize power usage

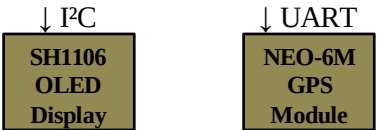
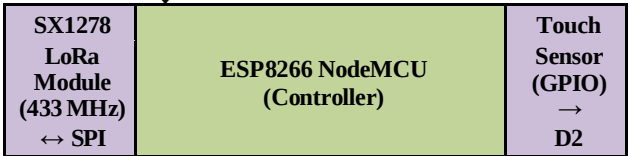
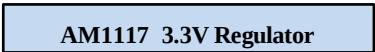
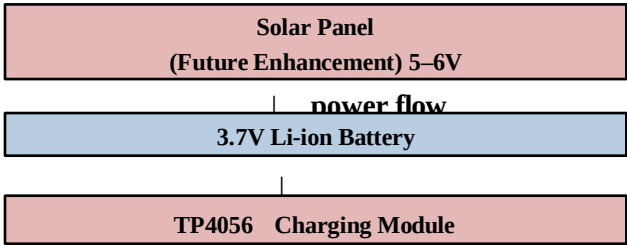
FUNTIONALBLOCKDIAGRAM:

- **Emergency Detection:** Touch sensor detects an emergency.
- **Data Processing:** Microcontroller processes the emergency signal.
- **Location Tracking:** GPS obtains the user's location.
- **LoRa Transmission:** Emergency message and location are transmitted through LoRa.
- **Relay Nodes:** Relay nodes forward the message to the destination.
- **OLED Display:** OLED displays system status and location information.



HARDWARE DESIGN

Sender Node (TerraLink)



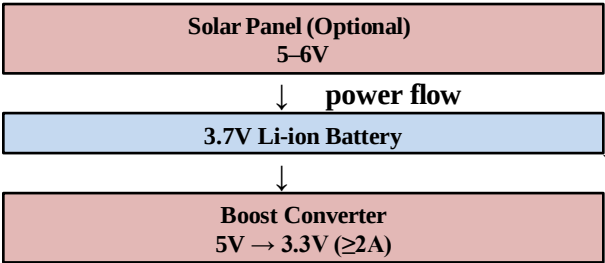
SX1278 ↔ ESP8266

Pin	ESP8266 Pin
VCC	3.3V
GND	GND
SCK	D5 (GPIO14)
MISO	D6 (GPIO12)
MOSI	D7 (GPIO13)
NSS	D1 (GPIO5)
RST	D0 (GPIO16)
DIO0	NC

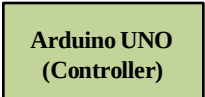
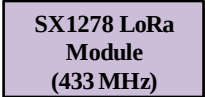
OLED / GPS ↔ ESP8266

Pin	ESP8266 Pin
OLED VCC	3.3V
OLED GND	GND
OLED SDA	D4
OLED SCL	RX
GPS VCC	VIN (5V)
GPS GND	GND
GPS TX	D3 (GPIO1)
GPS RX	NC

Relay Node (TerraLink)



↓ USB



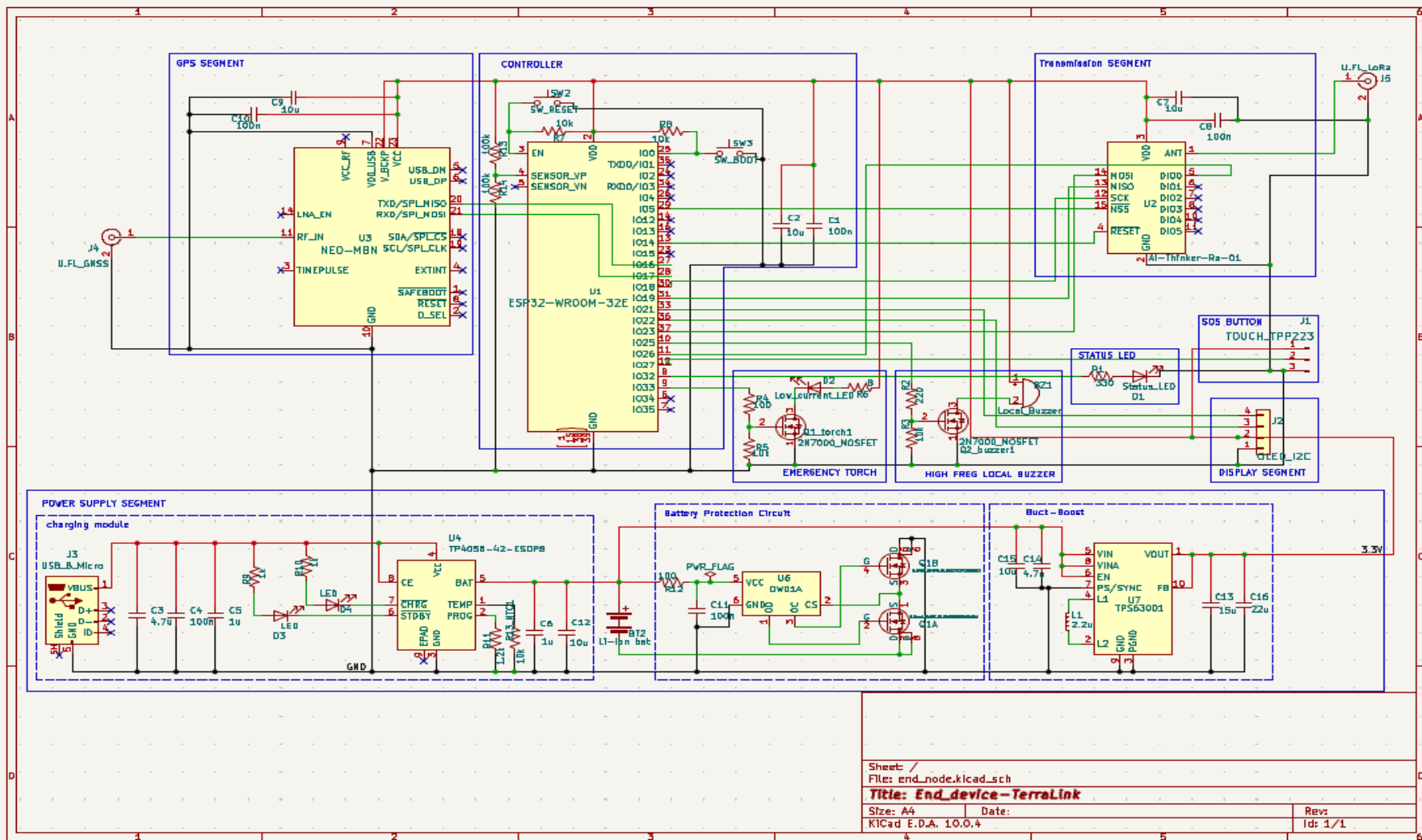
Pin	Arduino Pin
VCC	3.3V
GND	GND
SCK	D13
MISO	D12
MOSI	D11
NSS	D10
RST	D9
DIO0	2 (INT)

SX1278
LoRa
Module
(433
MHz)
SPI →

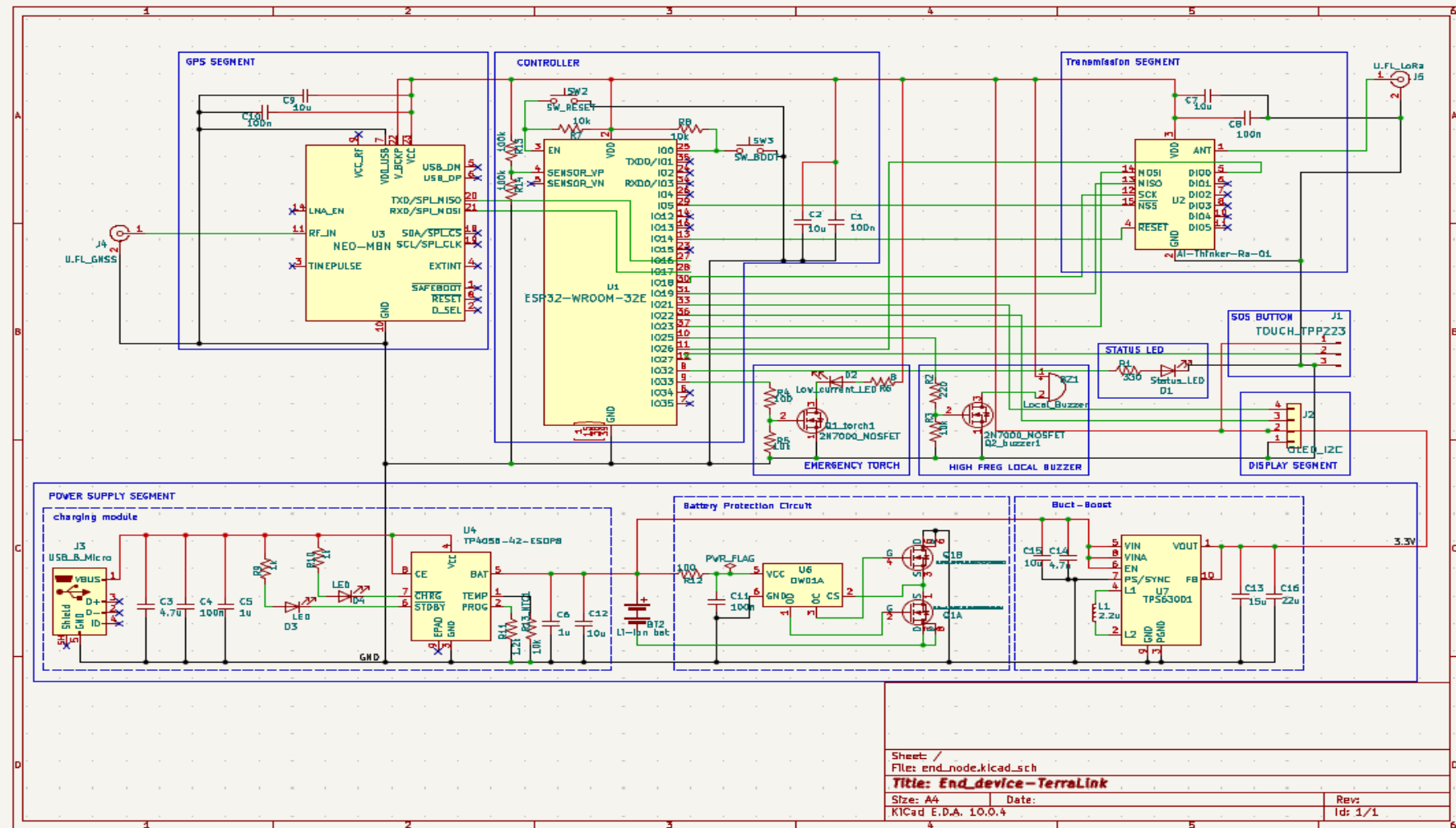


CIRCUIT SCHEMATIC:

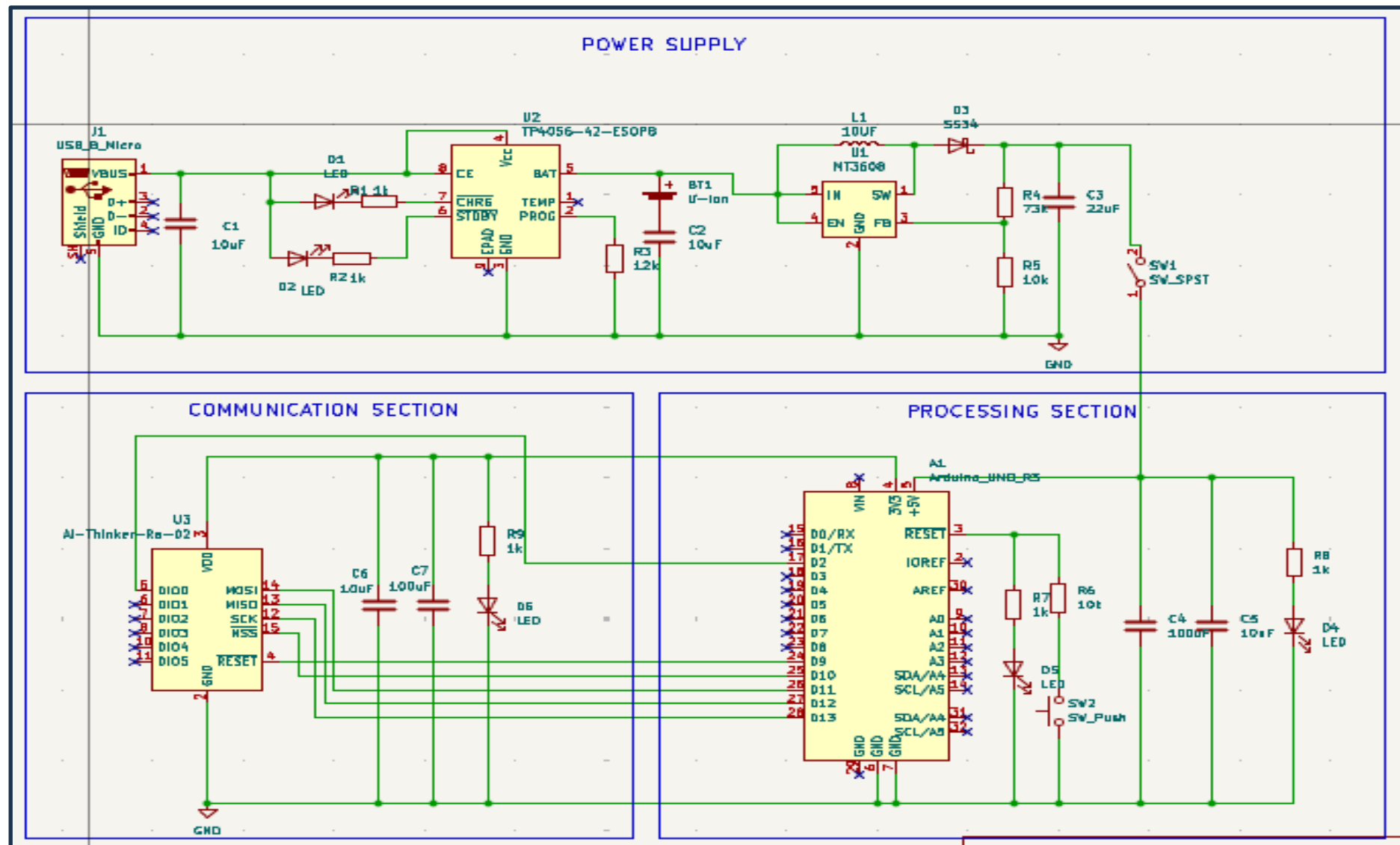
SENDER



RECEIVER



RELAY NODE



Algorithm: Dynamic Off-Grid Emergency Communication and Self-Healing

Routing

Input

- Local SOS/user event
- Incoming LoRa packet
- GPS location
- Operator command
- Node configuration
- Available neighbour/next-hop information

Output

- Delivered SOS/message
- Forwarded packet
- ACK / MSGACK / relevant response
- Updated incident/message status
- Recovered communication route when the current path becomes unavailable

1. System Initialization:

1. START

2. Initialize the microcontroller and required peripherals.

3. Initialize the LoRa communication module.

4. Initialize node-specific interfaces:

- Sender → GPS, OLED, SOS input, buzzer, controls
- Relay → LoRa interface and routing/packet processing
- RX1 → LoRa gateway and UART interface
- RX2 → keypad, UART, local command-center services

5. Initialize packet-processing state:

- Sequence information
- Packet/message identifiers
- Duplicate tracking
- Node configuration

6. Verify LoRa initialization.

7. IF LoRa initialization fails:

 Retry initialization.

8. ELSE:

 Enter continuous monitoring.

Continuous Monitoring:

9. Monitor the node continuously.
10. Check for:
 - Local SOS event
 - Local message/input
 - Incoming LoRa packet
 - Required response/acknowledgement
 - Routing/forwarding activity
 - Command-center request
11. IF a local event occurs:
Go to LOCAL PACKET GENERATION.
12. IF an incoming packet is received:
Go to PACKET RECEPTION AND VALIDATION.
13. Otherwise:
Continue monitoring.

2. Local SOS / Message Generation:

14. Detect local event.
15. Determine event type.
16. IF event is SOS:
Acquire or update GPS location.
17. Generate the required sequence/message identifier.
18. Construct the protocol packet.
19. Populate applicable packet fields:
 - SRC
 - DST
 - SEQ
 - TTL
 - HOP
 - NEXT
 - FROM
 - TYPE
 - MSGID
 - LAT
 - LON
 - MSG
20. Validate the generated packet.
21. Determine an appropriate next hop.
22. Check packet TTL.
23. IF TTL is invalid/expired:
Drop or terminate packet processing

24. ELSE:

Update the required hop/lifetime information.

25. Transmit the packet through LoRa.

26. Proceed to acknowledgement/response handling when required.

27. Return to monitoring.

3. Packet Reception and Validation:

28. Receive LoRa packet.

29. Parse the received packet.

30. Validate packet format and required fields.

31. IF packet is invalid:

Drop packet.

Return to monitoring.

32. Check packet identity using the relevant sequence/message identifier.

33. IF packet is already processed:

Treat it as a duplicate.

Drop packet.

Return to monitoring.

34. ELSE:

Record/process the packet identity.

35. Check TTL.

36. IF TTL has expired:

Drop packet.

Return to monitoring.

37. ELSE:

Update hop/lifetime information.

38. Check packet destination.

4. Destination Decision:

39. IF DST corresponds to the current node:

Process the packet locally.

Determine packet TYPE.

Execute the appropriate operation:

- SOS → emergency processing
- MSG → message processing
- ACK → acknowledgement processing
- MSGACK → message-delivery processing
- LOCREQ → location-request processing

- LOC → location-response processing
Generate the required response, if applicable.
Forward relevant information toward the
appropriate receiver/application layer.
Return to monitoring.

40. ELSE:

- The current node is an intermediate node
- Determine the next forwarding hop.
- Go to NEXT-HOP SELECTION.

5. Next-Hop Selection:

- 41. Examine available forwarding options.
- 42. Determine whether a usable next hop exists.
- 43. IF a valid next hop is available:
 - Set/select the next hop.
 - Forward the packet.
 - Continue response/forwarding handling.
- 44. IF no usable next hop is available:
 - Initiate dynamic route recovery.

6. Dynamic Self-Healing Route Recovery

- 45. Detect next-hop/link unavailability.
- 46. Mark the unavailable next hop as unusable
for the current forwarding operation.
- 47. Search for an alternative available neighbour/
forwarding node.
- 48. Alternative next hop available?
 - IF NO:
 - Retry according to the communication policy
or terminate/drop the packet when its forwarding
conditions cannot be satisfied.
 - IF YES:
 - Select the alternative next hop.
- 49. Update the forwarding information.
- 50. Re-check packet lifetime/TTL.
- 51. IF TTL remains valid:
 - Forward the packet through the alternative route.
- 52. ELSE:
 - Drop the packet.
- 53. Continue normal network operation.

7. Acknowledgement and Response Handling:

54. After a transmission requiring a response, monitor for the corresponding response.
55. Determine received response type.
56. IF valid ACK is received:
 - Confirm the relevant transmission.
57. IF valid MSGACK is received:
 - Mark the message as delivered/acknowledged.
58. IF a location response is received:
 - Process the returned location.
59. IF the expected response is not received within the applicable communication condition:
 - Consider retry or route-recovery processing where supported.
60. Return to normal packet/network processing.

8. Location Request and Response:

61. IF a location request is generated:
 - Identify the target node.
62. Generate the applicable location-request packet.
63. Forward the request through the LoRa network.
64. Target node receives the request.
65. Obtain available GPS/location information.
66. Generate the location response.
67. Return the location response through the network.
68. RX1 receives the location information.
69. Pass relevant information to RX2.
70. Update the Rescue Command Center.

9. RX1 Gateway Processing:

71. RX1 receives a valid LoRa packet.
72. Validate/process the packet.
73. Determine the packet type and relevant information.
74. Process:
 - SOS information
 - Messages
 - Acknowledgements
 - Location information

- Relevant network responses
75. Forward processed information to RX2 through the UART interface.
 76. Continue monitoring the LoRa network.

10. RX2 Rescue Command Center:

77. RX2 receives information from RX1.
78. Update the local command-center information.
79. Update applicable:
 - Incident information
 - Active SOS status
 - Message status
 - Node information
 - Network/communication status
80. IF an active SOS is received:
Prioritize the emergency incident.
81. Display/process the emergency information for the operator.
82. Check for an operator command.
83. IF no command is generated:
Continue monitoring.
84. IF an operator command/message is generated:
Create the appropriate outgoing packet/command.
85. Assign destination and required protocol information.
86. Send the command through the LoRa network.
87. Monitor the resulting response/acknowledgement.
88. Update the command/message delivery status.

11. Continuous Network Operation:

89. Complete the current packet/event operation.
90. Maintain relevant packet-processing state.
91. Continue monitoring incoming packets, local events and communication activity.
92. IF another event or packet occurs:
Repeat the corresponding processing branch.
93. ELSE:
Continue monitoring.
94. Repeat indefinitely while the node is operating.

START

1. Initialize MCU, LoRa and node-specific peripherals.
2. Initialize packet, sequence and duplicate-tracking state.
3. Verify LoRa initialization.
4. If initialization fails, retry; otherwise enter monitoring
5. Continuously monitor local events and incoming packets.
6. If a local SOS/message event occurs:
 - a. Acquire GPS information when required.
 - b. Generate sequence/message identifier.
 - c. Construct and validate the packet.
 - d. Select the next hop.
 - e. Check TTL.
 - f. If TTL is valid, update hop/lifetime information and transmit; otherwise drop the packet.
7. If a LoRa packet is received:
 - a. Validate packet format.
 - b. If invalid, discard it.
 - c. Check for duplicate packet identity.
 - d. If duplicate, discard it.
 - e. Check TTL.
 - f. If expired, discard it.
 - g. Update hop information.
 - h. Check destination.
8. If the current node is the destination:
 - a. Process the packet according to TYPE.
 - b. Handle SOS, MSG, ACK, MSGACK, LOCREQ or LOC as applicable.
 - c. Generate the required response.
9. If the current node is not the destination:
 - a. Determine the next forwarding hop.
 - b. If a valid next hop exists, forward the packet.
 - c. If the next hop is unavailable, initiate route recovery.
10. During route recovery:
 - a. Avoid the unavailable next hop.
 - b. Search for an alternative forwarding node.
 - c. If an alternative is available, update the next hop and forward the packet.
 - d. Otherwise retry or terminate forwarding according

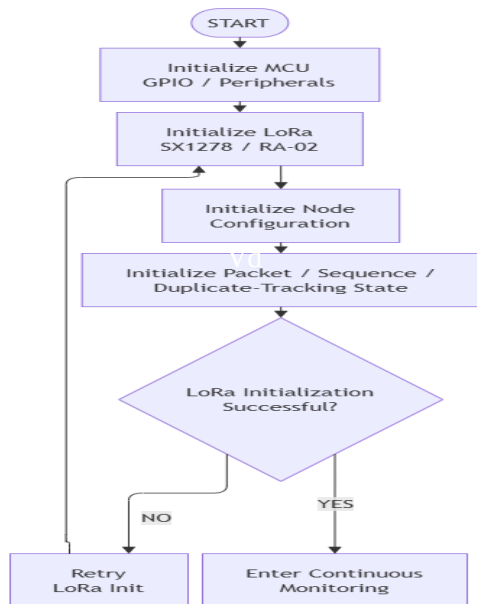
to the applicable condition.

11. Process required ACK/MSGACK/location responses.
12. RX1 processes received network information and passes relevant data to RX2.
13. RX2 updates the Rescue Command Center and handles operator-generated commands/messages.
14. Return to continuous monitoring.

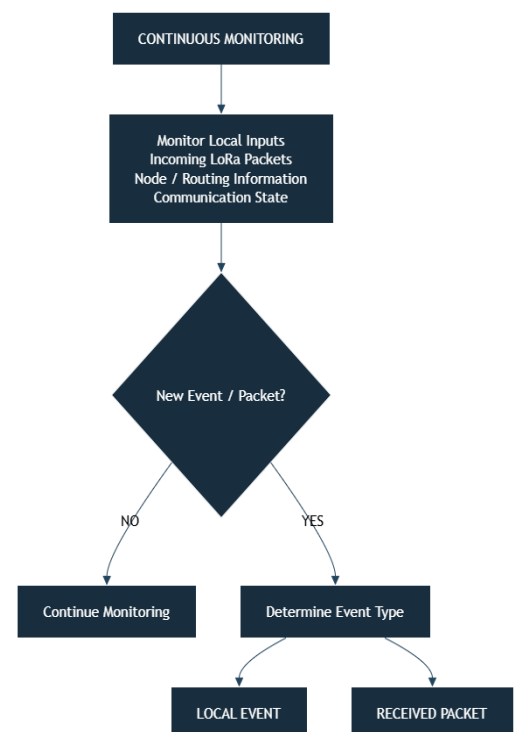
END

FLOWCHART:

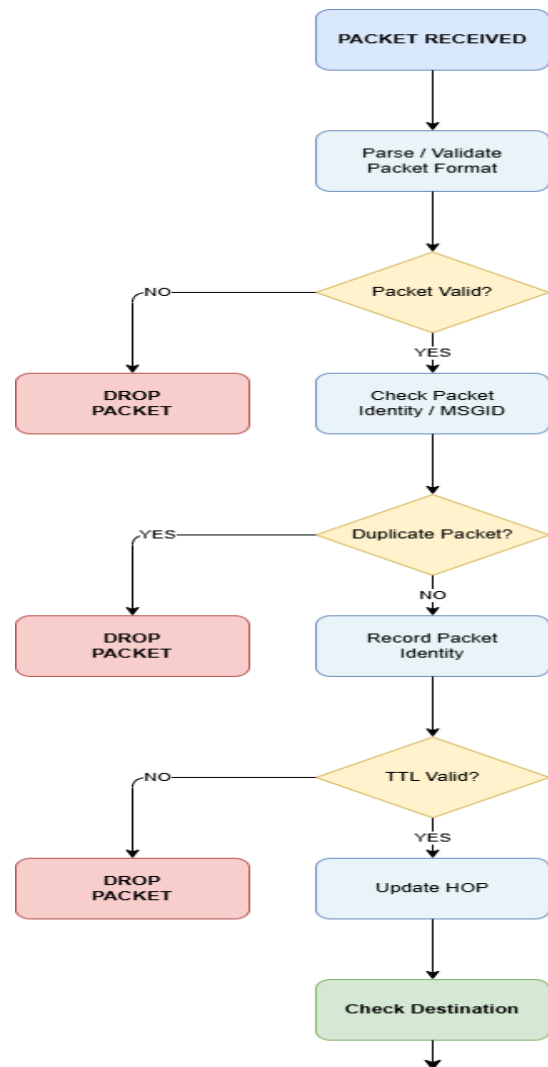
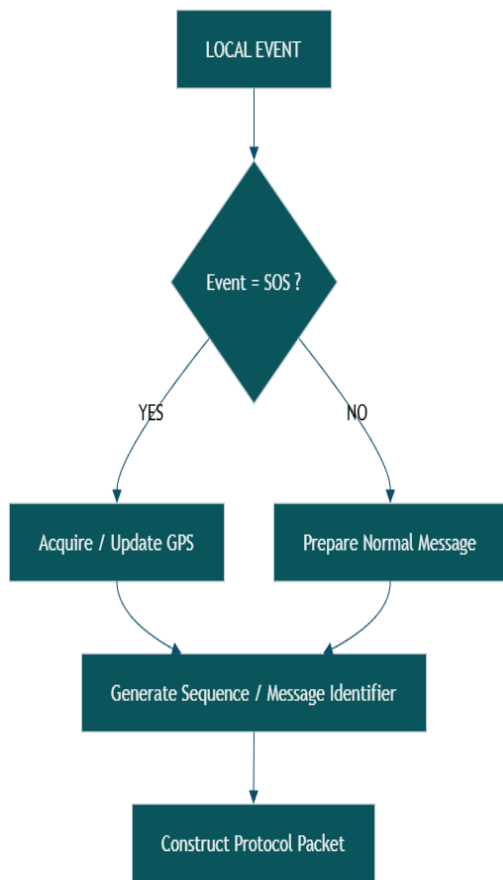
SECTION 1 — SYSTEM INITIALIZATION



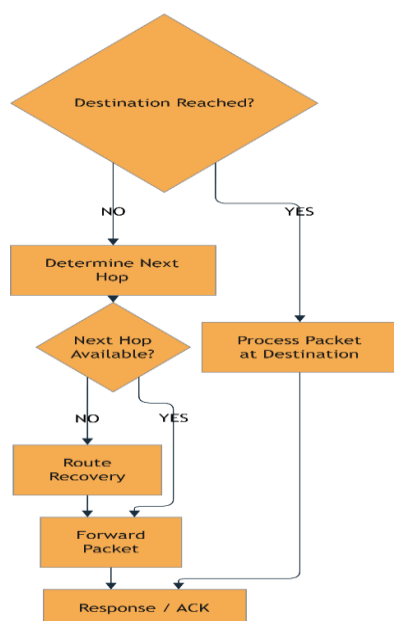
SECTION 2-CONTINUOUS MONITORING



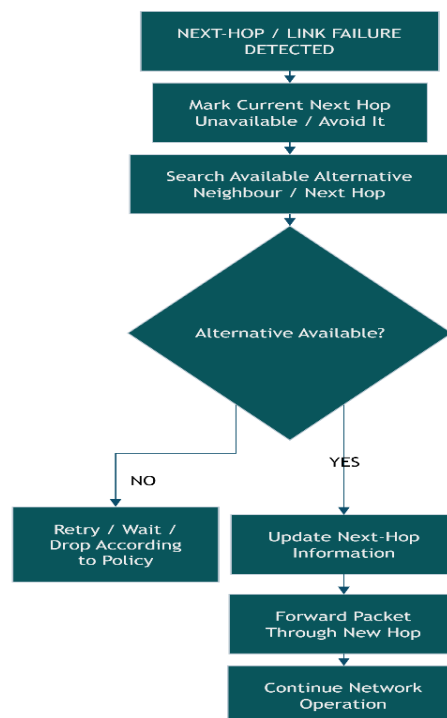
FORWARDING



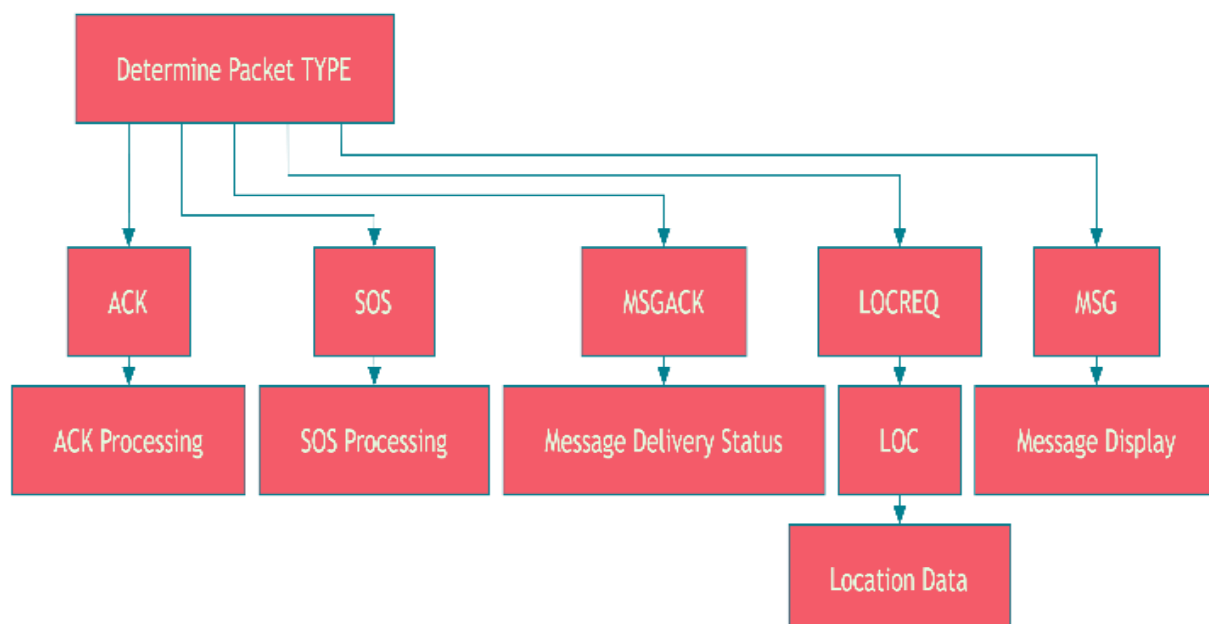
SECTION 5 — DESTINATION DECISION



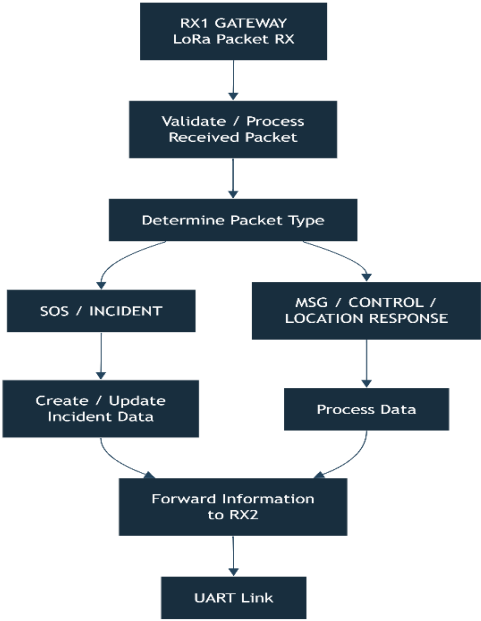
SECTION 6 — DYNAMIC SELF-HEALING ROUTING



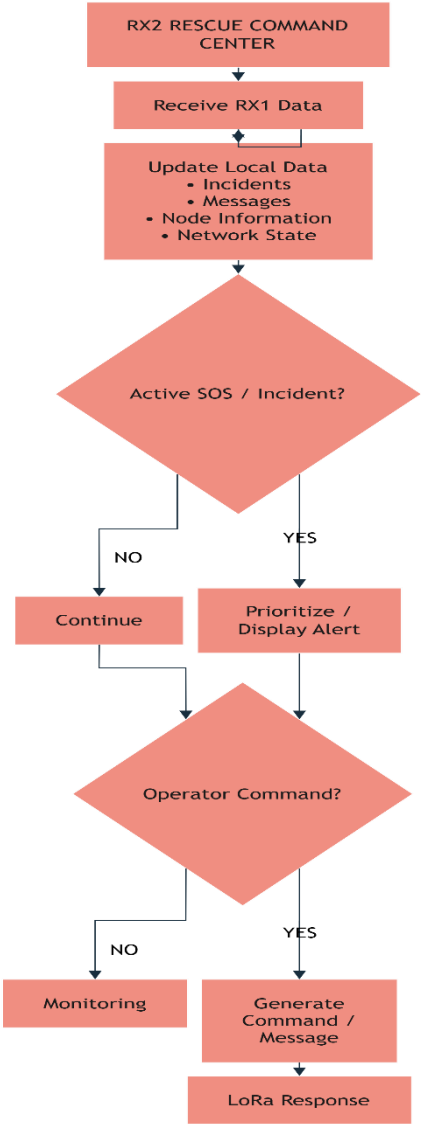
SECTION 7 — PACKET TYPE / RESPONSE HANDLING



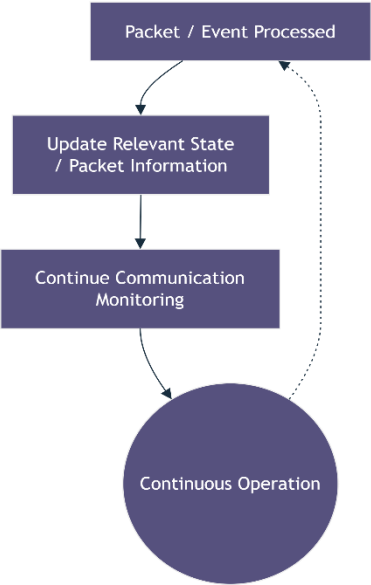
SECTION 8 — RX1 GATEWAY PROCESSING



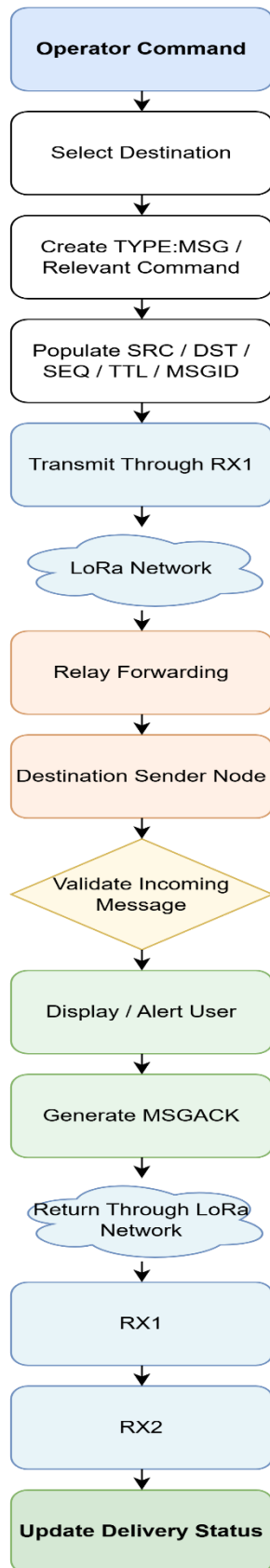
SECTION 9 — RX2 RESCUE COMMAND CENTER



SECTION 10— FINAL NETWORK LOOP

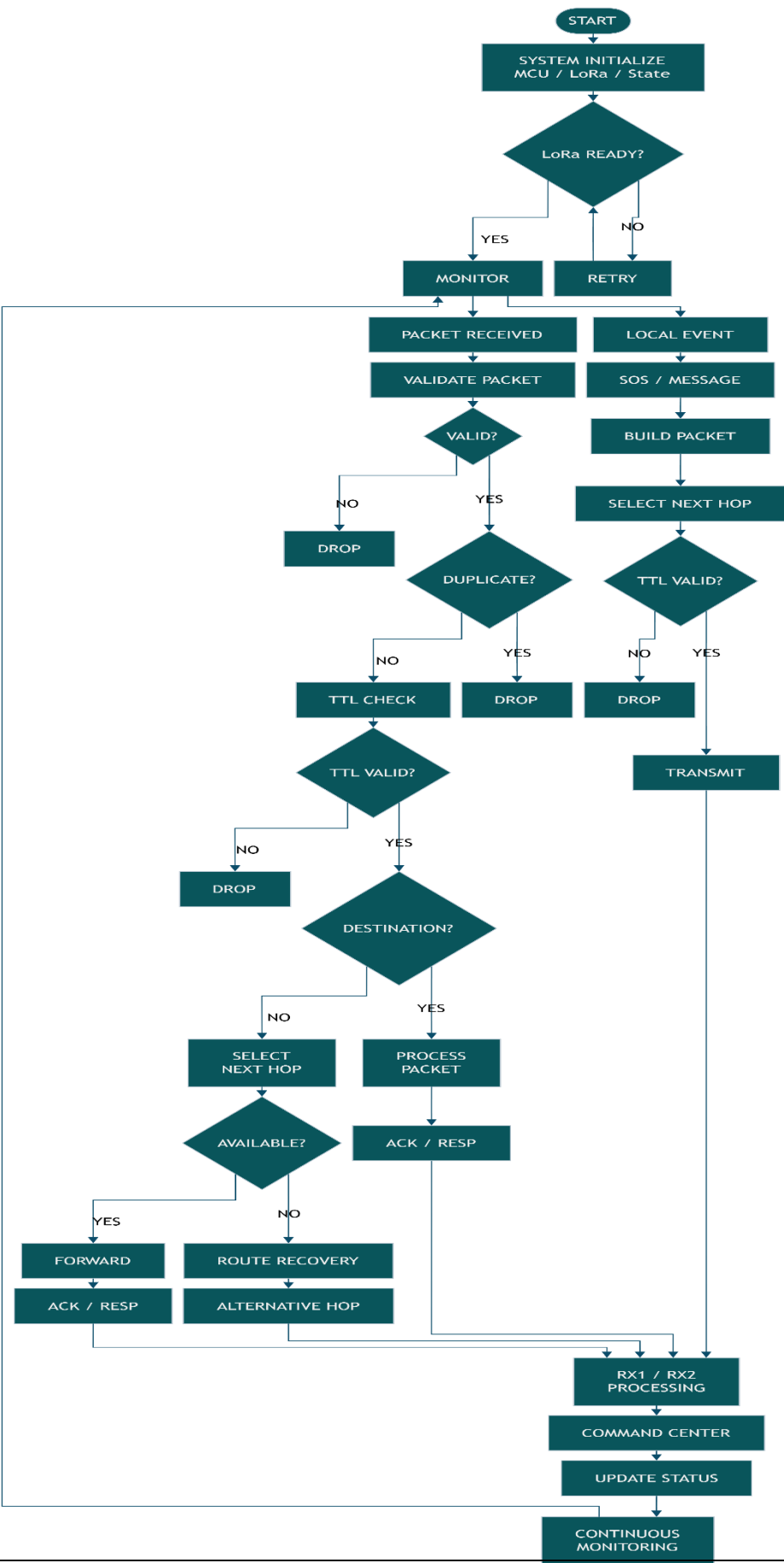


SECTION 11 — COMMAND / MESSAGE RETURN PATH



COMPLETE HIGH-LEVEL FLOW:

Dynamic TerraLink – System Operation, Packet Routing and Self-Healing Communication Flowchart

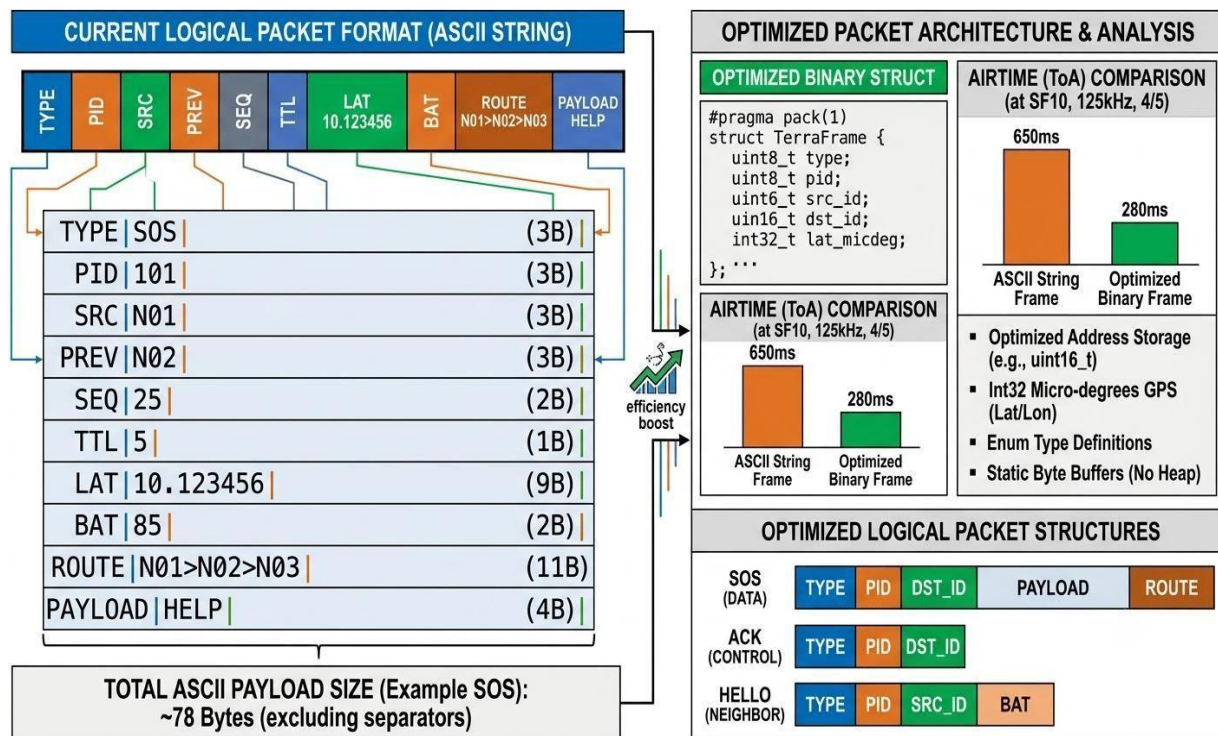


COMMUNICATION PROTOCOL

The Dynamic TerraLink Communication Protocol defines how an SOS packet is created, routed, validated, forwarded, acknowledged, and recovered when a communication link fails. It uses lightweight AODV-inspired routing, duplicate filtering, TTL-based loop prevention, hop-by-hop ACK, retransmission, and self-healing.

1. Packet Creation and Packet Structure

When an End Node detects an SOS event, it creates a structured packet containing the source, destination, sequence number, TTL, location, battery status, and emergency payload.



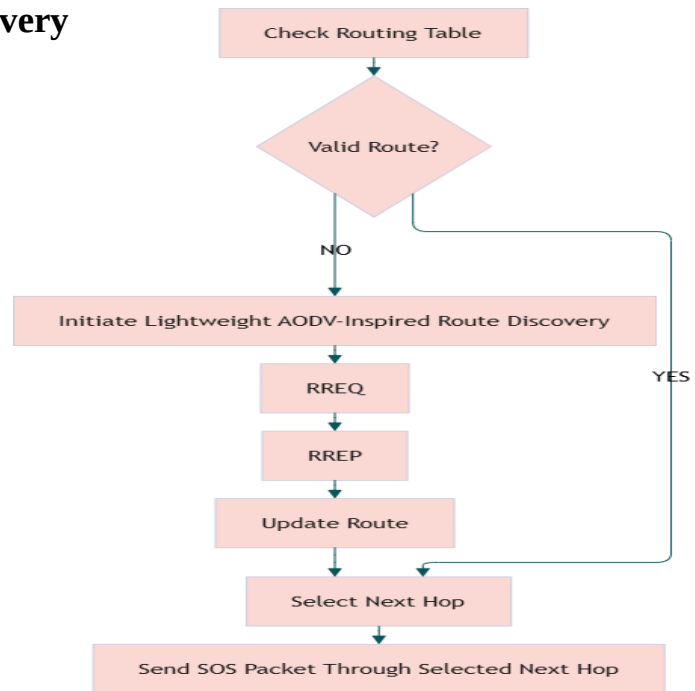
The main fields are:

- **TYPE** – Identifies the packet type.
- **PID** – Packet identifier.
- **SRC** – Source node.
- **DST** – Destination/base station.
- **SEQ** – Sequence number used for packet identification and duplicate filtering.
- **TTL** – Limits packet lifetime.
- **GPS** – Emergency location.
- **BATTERY** – Source-node battery status.
- **PAYLOAD** – Emergency information.

2. Route Selection and Route Discovery

Before transmitting an SOS packet, the node checks its routing table.

TerraLink uses a lightweight AODV-inspired route discovery mechanism rather than claiming full standard AODV implementation.

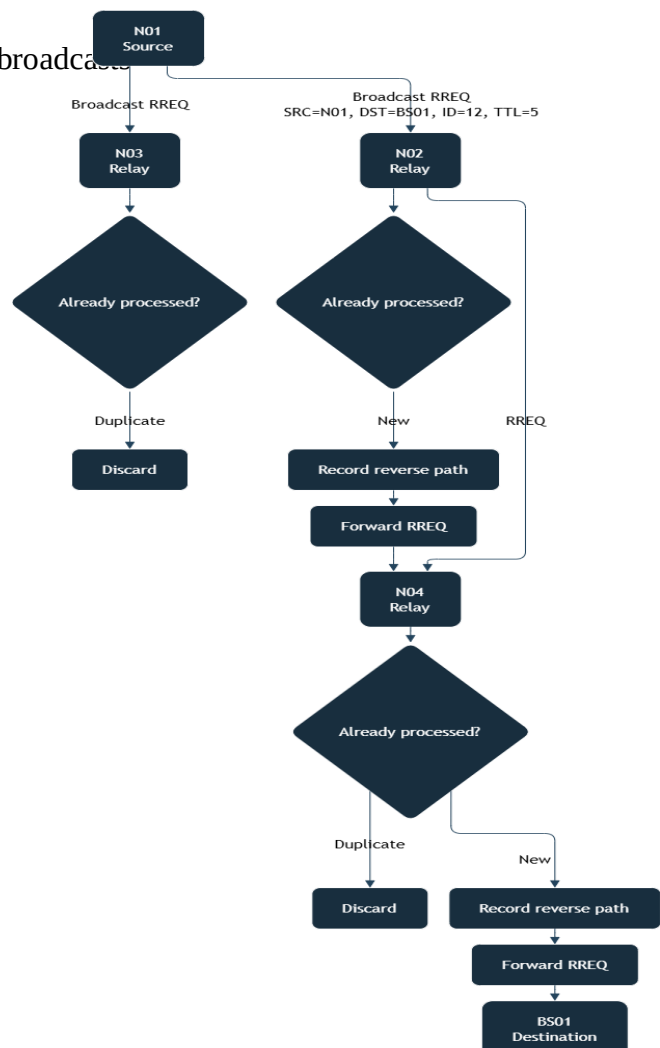


3. RREQ — Route Request

When there is no valid route, the source broadcasts an **RREQ**.

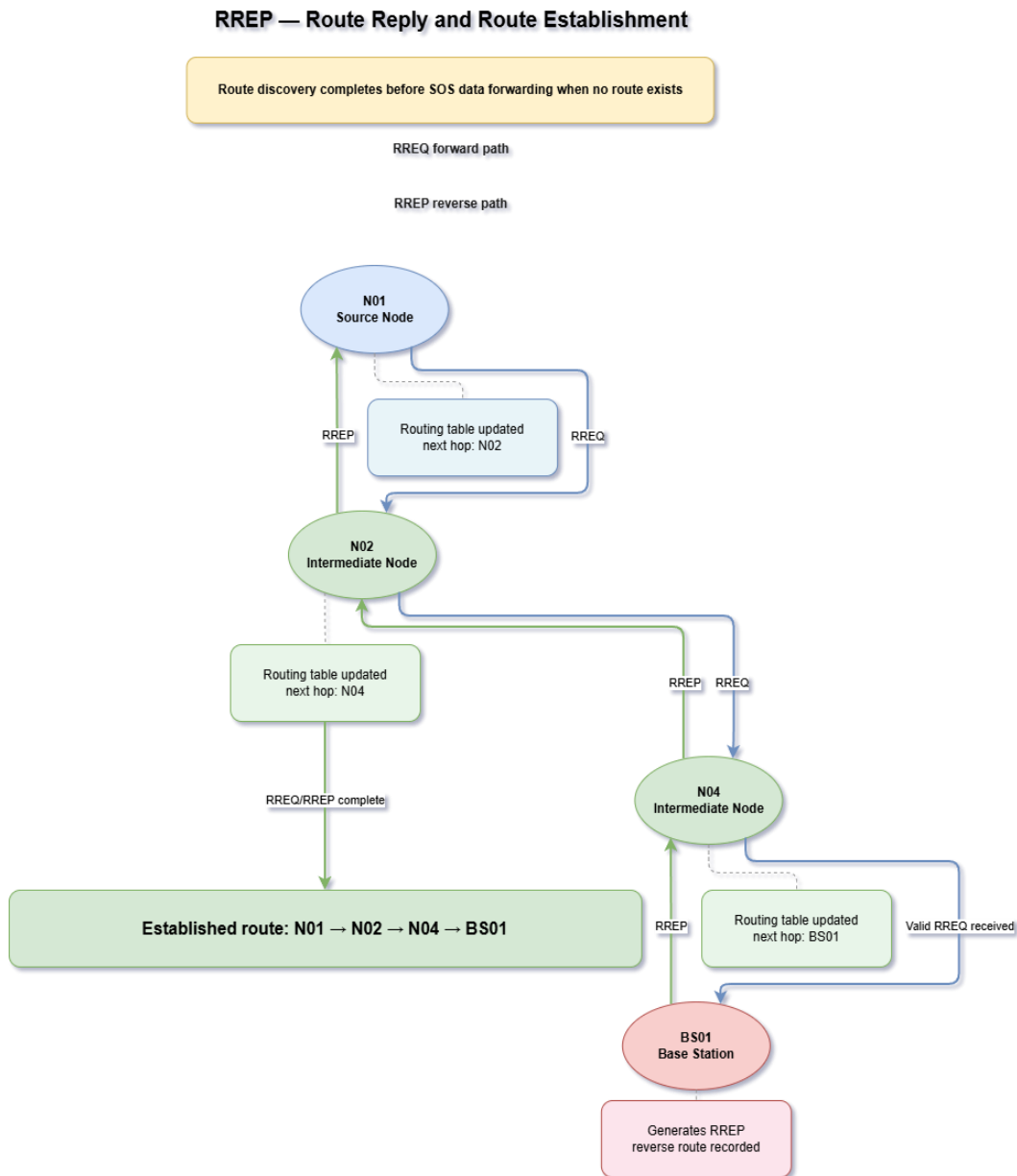
A relay receiving the RREQ:

1. Checks whether this RREQ has already been processed.
2. If duplicate → discard.
3. If new → record the reverse path.
4. Forward the RREQ toward the destination.



4. RREP — Route Reply

When the Base Station receives a valid RREQ, it generates an RREP. The RREP travels back through the reverse path:



The important point is that **RREQ/RREP establishes the route before the SOS data is forwarded when no route exists.**

5. Duplicate Filtering

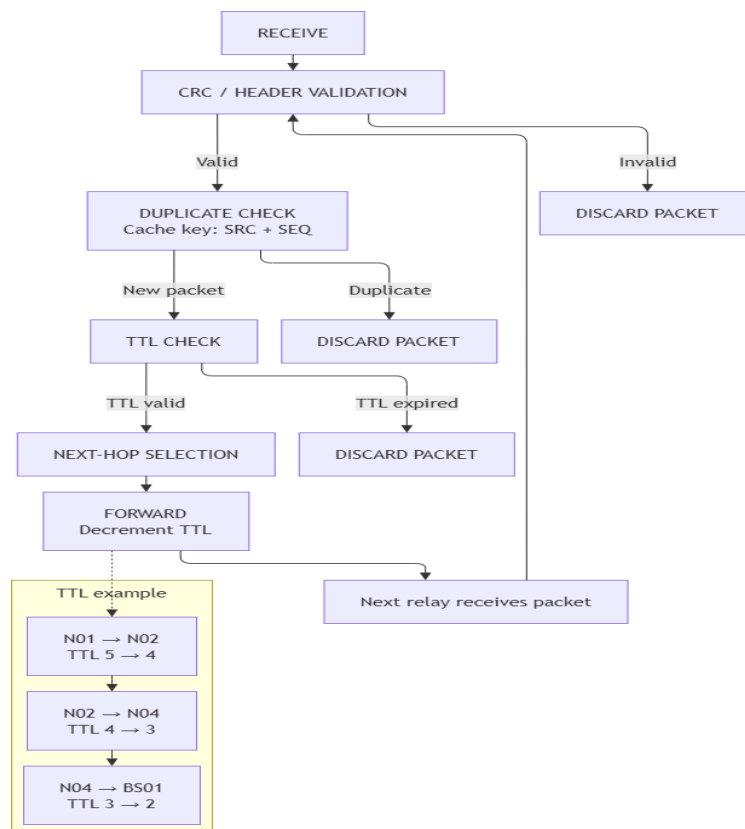
Each node maintains a small SRAM cache using:

SRC + SEQ

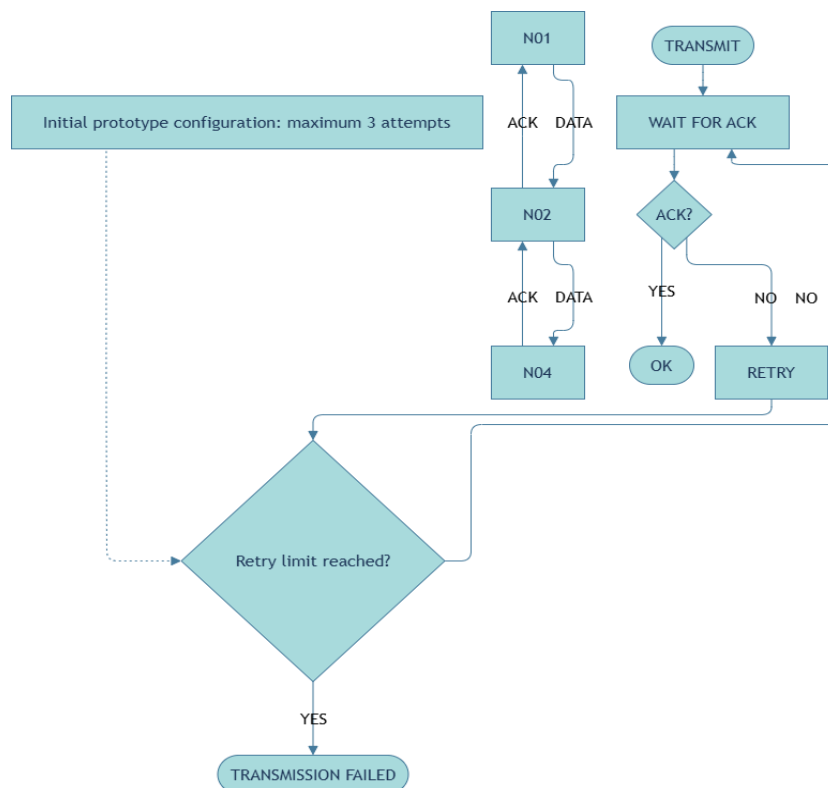
If the same source/sequence combination has already been processed, the packet is discarded to prevent repeated forwarding.

6. Packet Validation and Forwarding

Every relay validates a received packet before forwarding it. The processing order is:



7. Hop-by-Hop ACK and Retransmission

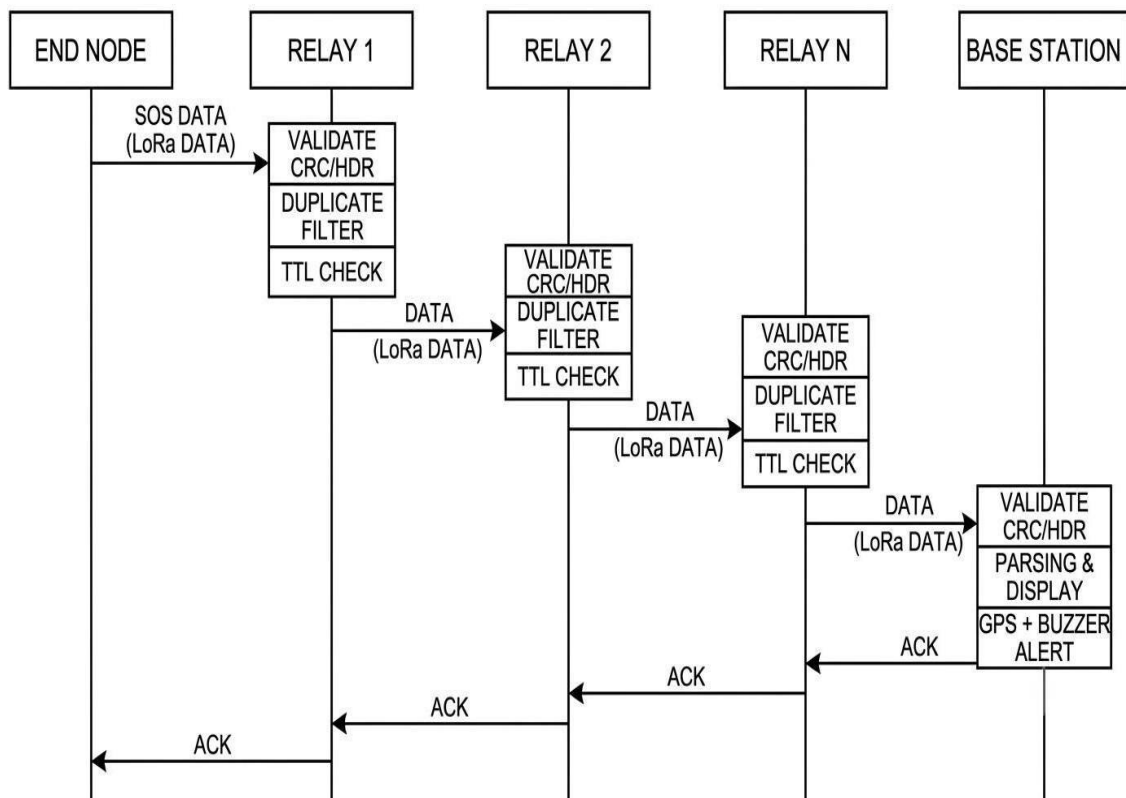
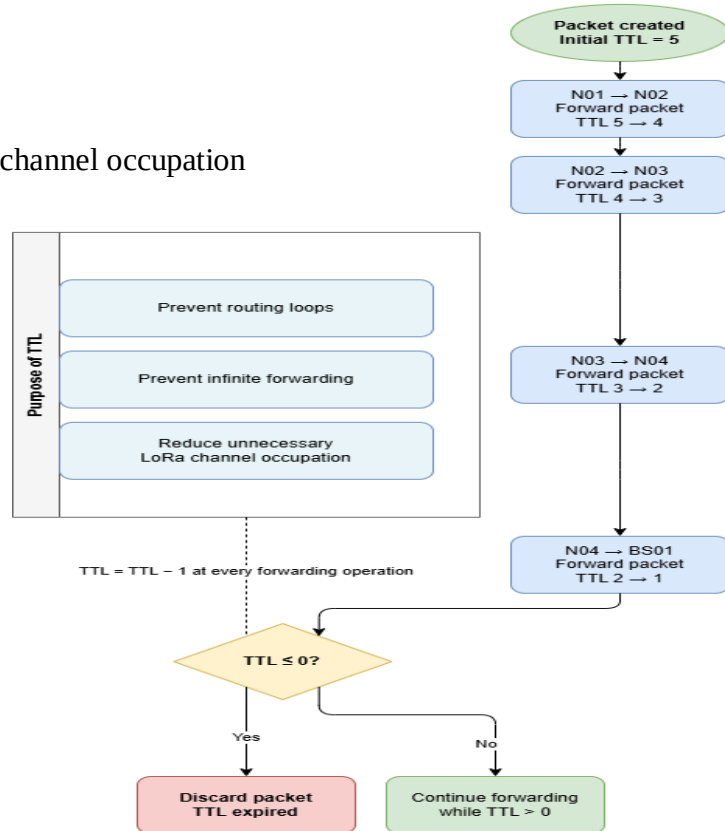


8. TTL — Packet Lifetime

TTL limits how many forwarding operations a packet can undergo.

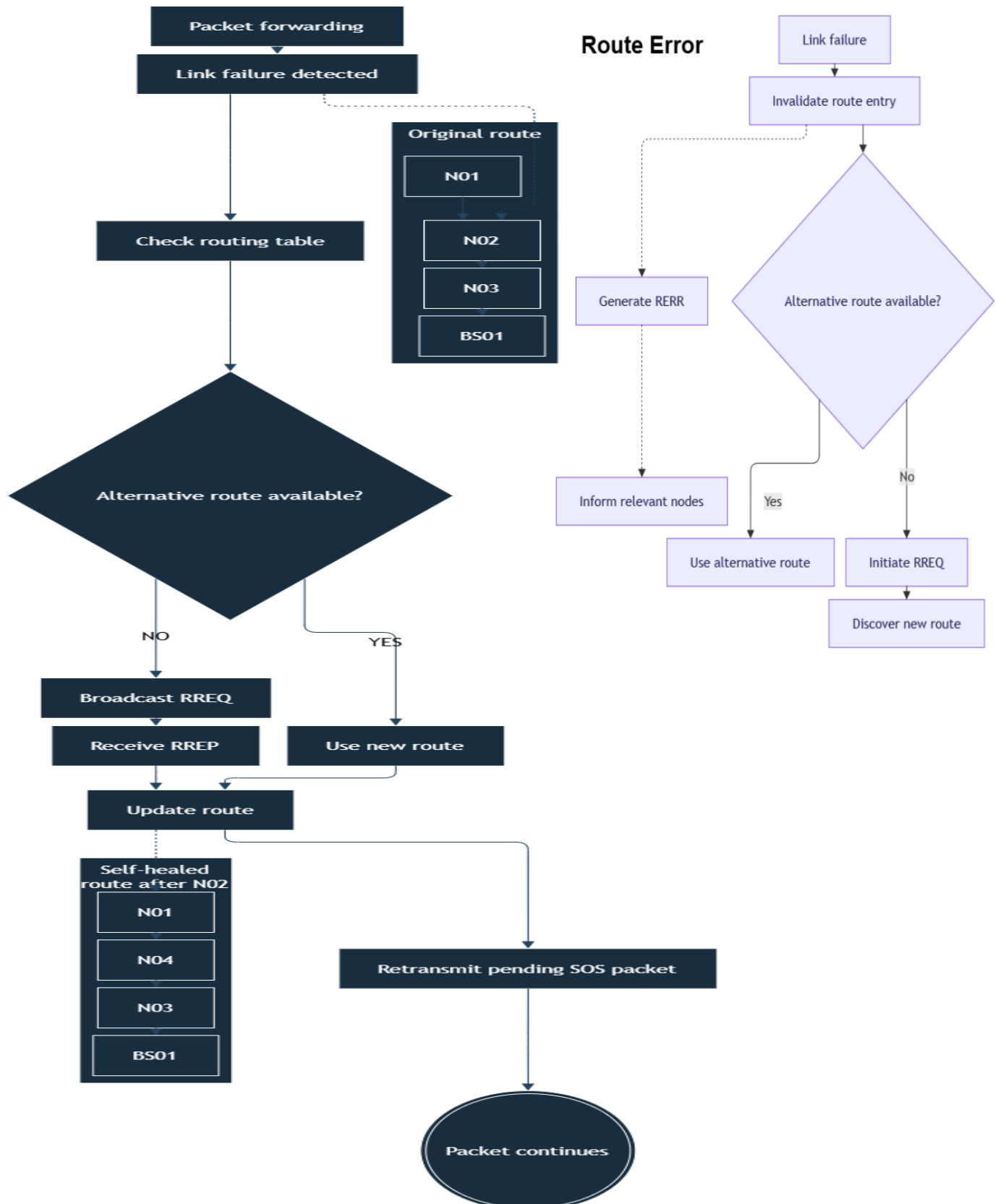
The purpose is to prevent:

- Routing loops
- Infinite forwarding
- Unnecessary LoRa channel occupation



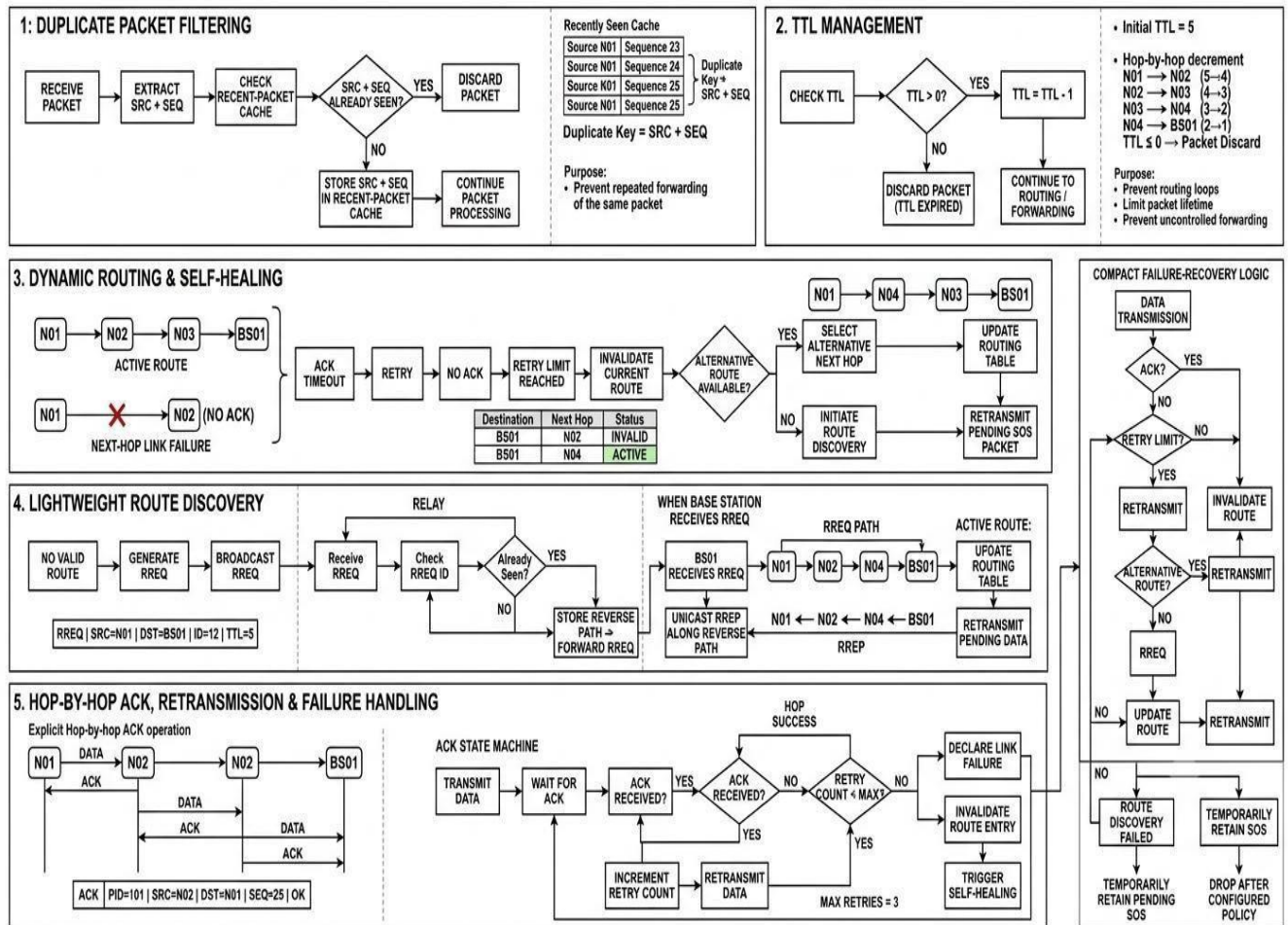
9. Self-Healing / Alternative Route

After detecting a failed next hop, the node checks whether another route is available.



DYNAMIC TERRALINK — COMMUNICATION PROTOCOL: ROUTING, TTL, ACK & SELF-HEALING

LoRa-Based Multi-Hop Emergency Communication



The TerraLink protocol has five core mechanisms:

- 1. Routing:** Find and select the next hop.
- 2. Validation:** Reject corrupted, duplicate, or expired packets.
- 3. Forwarding:** Move the packet hop-by-hop toward the Base Station.
- 4. Reliability:** Use ACK + limited retransmission.
- 5. Self-healing:** Detect failed links, discover an alternative route, update the routing table, and retransmit the pending SOS.

TerraLink combines **LoRa multi-hop forwarding**, **SRC+SEQ duplicate filtering**, **TTL-based loop prevention**, **hop-by-hop ACK/retransmission**, and **lightweight AODV-inspired route recovery** to provide resilient emergency-message delivery.